Přednáška 11: Zpracování sekvencí a revoluce Transformerů – NLP v materiálové vědě

Kapitola 1: Most z přednášky 10 – Chybějící dimenze dat (Sekvence)

Vítám vás na jedenácté přednášce. V celém tomto kurzu sledujeme systematický přístup k řešení problémů materiálové vědy pomocí AI. Naším hlavním tématem je, že volba architektury modelu není náhodná, ale je hluboce svázána s *fyzikální podstatou a strukturou našich dat*.

Pojďme si zrekapitulovat, kde se nacházíme, a podívat se na naši "svatou trojici" datových typů.

1. Data na mřížce (Grids): V Přednáška 9 jsme se naučili, že Konvoluční neuronové sítě (CNN) jsou mistry ve zpracování dat na euklidovské mřížce. Ať už jde o 2D pixely v obraze mikrostruktury nebo 3D voxely v tomografickém skenu, CNN jsou efektivní, protože jejich architektura obsahuje správné "induktivní předsudky" (inductive biases): princip *lokality* (konvoluční filtry) a *translační invarianci* (sdílení vah).

2. Relační data (Graphs): V Přednáška 10 jsme narazili na limit CNN: co když data nemají pravidelnou mřížku? Krystalová struktura není obraz, je to *graf* atomů (uzlů) propojených vazbami (hranami). Naučili jsme se, že Grafové neuronové sítě (GNN) řeší tento problém tím, že jejich architektura respektuje fundamentální *fyzikální symetrie* (permutační, rotační a translační invarianci) a operuje na principu *Message Passing* (předávání zpráv). Viděli jsme jejich klíčovou aplikaci jako motoru pro Strojově učené meziatomové potenciály (MLIPs).

Na samém konci přednášky 10 jsme si však položili otázku: Co nám ještě chybí? Co data, jejichž význam není dán ani polohou v mřížce, ani konektivitou v grafu, ale především **pořadím a kontextem v sekvenci**?

Toto je třetí a poslední klíčový datový typ, který musíme zvládnout. V materiálové vědě jsou tato data všudypřítomná a představují možná největší, dosud plně nevyužitý zdroj znalostí. Příklady, které jsme slíbili v přednášce 10 , jsou:

- **Spektra (XRD, Raman, EELS):** Difraktogram XRD je sekvence intenzit v závislosti na úhlu 2Θ. Význam píku (např. Braggův pík) není dán jedinou hodnotou, ale jeho tvarem, šířkou a pozicí *v kontextu* celé sekvence.

- **Časové řady:** Sledování vývoje energie, teploty nebo tlaku během MD simulace. Hodnota v čase t je silně závislá na hodnotách v t-1, t-2, ...

- **Vědecká literatura (NLP):** Nejbohatší a nejméně strukturovaný zdroj. Věta "Slitina Ti-6Al-4V byla žíhána při 800 °C" je sekvence slov (tokenů). Význam slova "800 °C" je dán *kontextem* celé věty – je to parametr syntézy.

Než se ponoříme do řešení, položme si otázku: proč na sekvence nemůžeme použít modely, které již známe?

- **Proč ne MLP (Přednáška 8)?** Vícevrstvý perceptron (MLP) vyžaduje "zploštění" vstupu do 1D vektoru. Tím bychom ztratili veškeré informace o *pořadí*. Navíc MLP vyžaduje vstup s pevnou délkou, ale sekvence (věty, spektra) mají proměnlivou délku.

- **Proč ne 1D CNN (Přednáška 9)?** Jednorozměrnou konvoluci *můžeme* použít a někdy se to dělá! Je skvělá pro zachycení *lokálních* vzorů (např. tvaru píku ve spektru). Ale stejně jako u obrazů má CNN *lokální receptivní pole*. Pro zachycení závislosti mezi slovem na začátku a na konci velmi dlouhé věty bychom potřebovali extrémně hlubokou CNN. Potřebujeme efektivnější model pro *dlouhodobé globální závislosti*.

Dnes se tedy podíváme na architektury navržené speciálně pro sekvenční data. Začneme historickým přístupem a ukážeme si, proč selhal, a poté přejdeme k revoluci, která pohání celou moderní AI.

Kapitola 2: "Klasické" řešení – Rekurentní Neuronové Sítě (RNN)

Princip (Jak?): Síť s "pamětí"

Intuitivní myšlenka Rekurentní Neuronové Sítě (RNN) je elegantní. Místo abychom zpracovali celou sekvenci najednou, zpracujeme ji krok po kroku, tak jako člověk čte větu. Váš mozek nezpracovává každé slovo izolovaně. Když čtete slovo "ho", uchováváte si v paměti kontext předchozích slov (např. "slitina Ti-6Al-4V vykazuje...").

RNN toto formalizuje pomocí dvou klíčových komponent:

1. **Smyčka (Loop):** Architektura, kde výstup z neuronové sítě v kroku t-1 je přiveden zpět jako *součást vstupu* pro krok t.
2. **Skrytý stav (Hidden State, h_t):** Toto je ten klíčový koncept. Je to vektor, který slouží jako **"paměť"** sítě. V každém kroku t model vezme aktuální vstup x_t (např. vektor pro slovo "ho") a skrytý stav z předchozího kroku h_t-1 (který kóduje "slitina Ti-6Al-4V vykazuje...") a zkombinuje je, aby vytvořil nový skrytý stav h_t.

Okamžitá limitace: Problémy s tréninkem

Jak jsme se naučili v přednášce 8 , modely trénujeme pomocí zpětné propagace a gradientního sestupu. Pro RNN musíme tento proces "rozbalit v čase" – technika se nazývá **Backpropagation Through Time (BPTT)**.

Když sekvenci "rozbalíme", RNN se efektivně stane *velmi hlubokou* neuronovou sítí, kde každá vrstva odpovídá jednomu časovému kroku a co je klíčové, *sdílí stejné váhy*. Toto sdílení vah vede ke dvěma zásadním problémům.

Limitace 1: Problém mizejících a explodujících gradientů

Při zpětné propagaci se gradient počítá pomocí řetězového pravidla, jak víme z přednášky 8. V BPTT to znamená, že gradient v časovém kroku t=1 je funkcí gradientu z kroku t=N. Aby se tam dostal, musí být opakovaně násoben maticí vah W (konkrétně její Jacobiho maticí).

- **Explodující gradienty:** Pokud jsou váhy ve W velké (singulární hodnoty > 1), gradient při opakovaném násobení exponenciálně roste. To vede k obrovským aktualizacím vah a trénink se stává nestabilním, často končí hodnotami NaN (Not a Number). Tento problém lze relativně snadno vyřešit technikou zvanou "gradient clipping" (ořezání gradientu).

- **Mizející gradienty (Větší problém):** Pokud jsou váhy ve W malé (singulární hodnoty < 1), gradient při opakovaném násobení exponenciálně *mizí* a blíží se nule.

Klíčový dopad mizejících gradientů: Gradient pro slovo na začátku sekvence (např. "Slitina") se po zpětné propagaci z konce dlouhé věty stane nulovým. Model se *nikdy nenaučí* dlouhodobé závislosti. Nedokáže spojit slovo "jí" ve větě "... proto je slitina tak křehká, že *jí* nelze použít" se slovem "slitina" o 15 slov dříve.

Limitace 2: Nemožnost paralelizace (Sekvenční "bottleneck")

I kdybychom problém gradientů ignorovali, zůstává fundamentální architektonický problém. Výpočet skrytého stavu h_t *ze své definice* vyžaduje výstup h_t-1.

- **Důsledek:** Nemůžeme zpracovat všechna slova ve větě najednou. Musíme čekat. Výpočet pro slovo "je" musí počkat na dokončení výpočtu pro slovo "slitina".

- Pro moderní hardware, jako jsou GPU a TPU, které jsou navrženy pro masivní paralelní výpočty (jak jsme viděli u CNN v přednášce 9), je toto nejhorší možný scénář. Většina čipu zůstává nevyužitá, zatímco čeká na dokončení sekvenčního kroku.

- Trénink RNN na dlouhých sekvencích je *extrémně* pomalý.

Vylepšení: LSTM a GRU (Krátká historická odbočka)

Na problém mizejících gradientů bylo v 90. letech navrženo geniální řešení: **Long Short-Term Memory (LSTM)** a později zjednodušená varianta **Gated Recurrent Unit (GRU)**.

- **Princip:** Místo jediného "skrytého stavu" má LSTM i "stav buňky" (cell state) – jakýsi oddělený paměťový kanál, který si lze představit jako dopravníkový pás.

- Tok informací do a z této paměti je řízen malými neuronovými sítěmi zvanými **"brány" (gates)** :
    1. **Forget Gate (Brána zapomnění):** Rozhodne, co ze staré paměti (cell state) je třeba vymazat.
    2. **Input Gate (Brána vstupu):** Rozhodne, jaké nové informace z aktuálního vstupu se uloží do paměti.
    3. **Output Gate (Brána výstupu):** Rozhodne, co z paměti se použije pro aktuální skrytý stav h_t.

- **Jak řeší gradienty:** Tím, že "cell state" prochází skrze síť s minimálními transformacemi (pouze násobení a sčítání řízené branami), gradient může protékat "nezraněn" zpět v čase. To elegantně vyřešilo problém mizejících gradientů a umožnilo modelům učit se dlouhodobé závislosti.

Klíčový vhled: Skutečný zabiják RNN

Takže LSTM/GRU jsou řešením, že? Problém vyřešen.

Bohužel ne.

Vyřešily *jen* problém mizejících gradientů. **Problém nemožnosti paralelizace zůstává!**

Výpočet v čase t stále závisí na čase t-1. LSTM je ve skutečnosti *ještě pomalejší* než klasická RNN kvůli všem těm výpočtům bran v každém kroku.

V éře velkých dat a masivního paralelismu (GPU) se ukázalo, že *skutečným úzkým hrdlem* nebyla matematika gradientů, ale *sekvenční povaha* výpočtu. Svět potřeboval architekturu, která by se zbavila rekurence úplně.

Kapitola 3: Moderní řešení – Transformery a Mechanismus Pozornosti

Jádro: Architektura bez rekurence

V roce 2017 byl publikován přelomový článek s názvem "Attention Is All You Need". Tento název je výstižný. Představil architekturu **Transformer**, která se *zcela zbavuje* rekurzivních smyček.
Místo toho se spoléhá *pouze* na mechanismus zvaný **"Self-Attention" (Sebe-Pozornost)** k modelování vztahů mezi všemi slovy v sekvenci.

Klíčový koncept: Mechanismus "Self-Attention"

- **Intuitivní vysvětlení:** Představte si větu: "Slitina byla *křehká*, protože obsahovala *intermetalickou* fázi."
  - Pro pochopení významu slova "křehká" se RNN musí spolehnout na to, že se tato informace nějak "protlačila" skrytým stavem přes 4 slova.
  - Self-Attention umožňuje modelu "podívat se" na *všechna* ostatní slova v sekvenci *najednou*. Pro slovo "křehká" se model naučí (pomocí vah), že má věnovat vysokou "pozornost" slovu "intermetalickou", i když jsou od sebe daleko. Model se naučí, že tato slova spolu kontextově souvisejí.

- **Technické vysvětlení (Analogie s databází):** Self-Attention pro každý vstupní token (slovo) vypočítá tři různé vektory. Dělá to tak, že vezme vstupní vektor x_i a vynásobí ho třemi různými maticemi vah (W_Q, W_K, W_V), které se naučí během tréninku. Tím získá:
  1. **Query (Dotaz) Q:** "Já jsem toto slovo. Hledám kontext." (Např. Q_křehká)
  2. **Key (Klíč) K:** "Já jsem *nějaké* slovo v sekvenci. Toto je můj 'popisek', co nabízím." (Např. K_slitina, K_byla, K_intermetalickou)
  3. **Value (Hodnota) V:** "Já jsem *to samé* slovo. Pokud je můj 'Klíč' relevantní pro tvůj 'Dotaz', zde je *skutečná informace/obsah*, kterou ti předám." (Např. V_slitina, V_byla, V_intermetalickou)

- **Proces (zjednodušeně):**
  1. **Výpočet skóre:** Pro naše slovo i (např. "křehká") vezmeme jeho dotaz Q_i a vypočítáme "skóre podobnosti" skalárním součinem s *každým* klíčem K_j v celé sekvenci. Skóre_ij = Q_i · K_j.
  2. **Normalizace (Softmax):** Tato skóre (která mohou být libovolně velká či malá) převedeme na váhy pozornosti w_ij pomocí funkce Softmax. Tím zajistíme, že váhy jsou kladná čísla a jejich součet přes celou větu je 1. (Vysoké skóre Q_křehká · K_intermetalickou se přemění na vysokou váhu, např. 0.8).
  3. **Vážený průměr:** Výstupní reprezentace y_i pro slovo i je pak *vážený průměr* všech Value vektorů (V_j) v celé sekvenci, kde váhy jsou w_ij, které jsme právě spočítali.
  - Model se tyto váhy v maticích W_Q, W_K, W_V naučí během tréninku, aby vytvářel relevantní kontextové reprezentace.

Výhody Transformerů (Přímá odpověď na limity RNN)

Tato architektura geniálně řeší oba fundamentální problémy RNN:

1. **Řešení problému dlouhodobých závislostí:**
   - V RNN byla cesta signálu mezi slovem 1 a slovem N dlouhá O(N) kroků, což vedlo k mizejícím gradientům.
   - V Transformeru je cesta signálu mezi libovolnými dvěma slovy *přímá*. Model přímo počítá interakci (Q_1 · K_N). Délka cesty je O(1). Problém mizejících gradientů vlivem délky sekvence je *eliminován*.

2. **Řešení problému paralelizace:**
    - V RNN jsme museli výpočet provádět krok za krokem N-krát.
    - V Transformeru neexistuje *žádná* časová závislost. Výpočty Q, K, V pro všechna slova a následné výpočty pozornosti (které jsou jen násobením matic) lze provést *najednou* a *paralelně* pro celou sekvenci.
    - **Klíčový vhled:** Právě tato schopnost paralelizace je důvodem, proč můžeme trénovat *obrovské* modely (miliardy parametrů) na *obrovských* datasetech (celý internet). RNN by se na takovém hardwaru (GPU/TPU) "nudily" a trénink by trval tisíce let.

Pro rekapitulaci si shrňme klíčové rozdíly v tabulce.

Vlastnost	"Vanilla" RNN	LSTM / GRU	Transformer (Self-Attention)
Základní princip	Rekurentní smyčka	Rekurentní smyčka s "branami" (gates)	Self-Attention (bez rekurence)
Zpracování sekvence	Sekvenční (krok po kroku)	Sekvenční (krok po kroku)	Paralelní (celá sekvence najednou)
Rychlost tréninku	Pomalá	Velmi pomalá (více výpočtů na krok)	**Extrémně rychlá** (vysoce paralelizovatelná)
Problém mizejících gradientů	Ano (velký problém)	**Vyřešeno** (díky branám)	Není relevantní (jiná architektura)
Zachycení dlouhodobých závislostí	Velmi špatné	Dobré	**Vynikající** (cesta signálu O(1))

Architektura: Zmínka o GPT-2

Transformer je obecný stavební blok. Existují různé architektury, které ho využívají (např. BERT, GPT). Slavným příkladem, který odstartoval moderní éru velkých jazykových modelů, je **GPT-2** (Generative Pre-trained Transformer 2).

Architektura GPT-2 je v podstatě "svazek" (stack) 12 nebo více Transformer "decoder" bloků (přesněji "masked decoder" bloků). Funguje *autoregresivně*: pro generování textu vezme vstupní sekvenci, zpracuje ji, predikuje *jedno* další slovo (token), připojí toto nové slovo k sekvenci a celý proces opakuje. Právě tato architektura, škálovaná na obrovská data, se naučila generovat koherentní a kontextově relevantní text.

Kapitola 4: Klíčová strategie – Transfer Learning a Aplikace v NLP (Natural Language Processing)

Strategie: Transfer Learning – Proč nemusíme trénovat vlastní GPT

Trénink modelu jako GPT-2 nebo BERT od nuly stojí miliony dolarů a vyžaduje masivní výpočetní zdroje a petabajty dat. Pro 99 % vědeckých aplikací je toto nerealistické.

Naštěstí to *není nutné* díky dominantnímu paradigmatu moderního hlubokého učení: **Transfer Learning (Přenosové učení)**.

Princip: Dvoufázový proces "Předtrénuj a Dolaď"

Myšlenka je jednoduchá: Využijeme znalosti, které se model naučil na obecné úloze, a přeneseme je na náš specifický problém.

Fáze 1: Předtrénování (Pre-training)

- **Kdo:** Velké společnosti (Google, OpenAI, Meta).
- **Co:** Vezmou obrovskou architekturu (Transformer) a trénují ji na *masivním, obecném, neanotovaném* datasetu (např. celá Wikipedie, miliony knih, miliardy webových stránek).

- **Jak:** Úloha je **"Self-Supervised"** (sebe-řízená). Model nepotřebuje lidské anotace, učí se "sám z dat". Běžné úlohy jsou:
    1. **Masked Language Modeling (MLM)** (používá BERT): Z textu se náhodně odstraní 15 % slov (nahradí se tokenem ``) a model se trénuje, aby *predikoval, jaká slova chybí*, na základě okolního kontextu.
    2. **Next Token Prediction** (používá GPT): Model se trénuje, aby na základě všech předchozích slov predikoval *následující slovo*.

- **Výsledek:** Předtrénovaný model. Není to model pro žádnou konkrétní úlohu, ale je to "digitální mozek", který se naučil "rozumět" jazyku, gramatice, sémantice a dokonce i obecným faktům o světě (např. že "ocel" je "tvrdá" nebo že "Paříž" je "hlavní město Francie").

Fáze 2: Dolaďování (Fine-tuning)

- **Kdo:** My (vědci v materiálové vědě).
- **Co:** Vezmeme tento hotový, veřejně dostupný předtrénovaný model.

- **Jak:** "Odstřihneme" jeho poslední vrstvu (která byla pro pre-tréninkovou úlohu) a nahradíme ji novou, malou vrstvou pro *naši specifickou úlohu* (např. klasifikátor). Poté model "dotrénujeme" (mírně upravíme všechny jeho váhy) na naší *malé, specifické, anotované* datové sadě (např. 10 000 abstraktů s anotacemi, zda popisují syntézu, charakterizaci, nebo simulaci).

- **Výsledek:** Vysoce výkonný, specializovaný model pro náš úkol, získaný za zlomek výpočetních nákladů.

Aplikace (NLP): Těžba dat z vědecké literatury

- **Problém:** V materiálové vědě publikujeme desítky tisíc článků ročně. Žádný člověk ani tým je nedokáže přečíst. Znalosti o syntézních postupech, parametrech a vlastnostech jsou "uzamčeny" v nestrukturovaném textu (PDF).
- **Řešení:** Použití modelů NLP (Transformerů) natrénovaných pomocí Transfer Learningu k automatickému "čtení" a *extrakci* těchto informací do strukturované databáze.
- Příklad úlohy: Trénujeme model, aby v textu našel a označil entity jako **MATERIÁL**, **VLASTNOST** a **HODNOTA**. (Např. "Zjistili jsme, že slitina XYZ má pevnost v tahu 1200 MPa".)

Nástroje a modely pro materiálovou vědu

- **Hugging Face:**
  - Toto není jen knihovna, je to *ekosystém*. Je to v podstatě "GitHub pro AI modely".
  - **Knihovna transformers:** Open-source Python knihovna, která umožňuje na pár řádků kódu stáhnout (Fáze 1) a dolaďovat (Fáze 2) tisíce různých předtrénovaných modelů.
  - **Model Hub:** Centrální repozitář, kde komunita sdílí modely a datasety.
  - Hugging Face zásadně demokratizoval přístup k velkým jazykovým modelům.

- **MatBERT / MaterialBERT:**
  - Toto je náš klíčový příklad! Je to přímá aplikace Transfer Learningu *pro naši doménu*.
  - **Co to je:** Je to model architektury BERT, který *nebyl* předtrénován na obecné Wikipedii, ale byl *předtrénován od nuly* (nebo dále dotrénován) na milionech abstraktů a článků z materiálové vědy.

  - **Klíčový vhled (Proč je lepší?):** Důvodem je *slovník (tokenizer)* a *kontext*.

    1. **Tokenizer:** Běžný BERT tokenizer by slovo "XRD" nebo slitinu "Ti-6Al-4V" rozsekal na nesmyslné pod-jednotky ("Ti", "-", "6", "Al",...), protože je nikdy neviděl. Tomu se říká "tokenizer effect". MatBERT tokenizer byl vytvořen na materiálových textech, takže "XRD" a "Ti-6Al-4V" jsou pro něj *jednotlivé, smysluplné tokeny*.

    2. **Kontext:** Během předtrénování se MatBERT naučil, že slovo "mez" se v našem kontextu obvykle vyskytuje se slovem "kluzu" nebo "pevnosti", zatímco běžný BERT by ho znal jen ve smyslu "mez stromů". * **Výsledek:** Při dolaďování na specifickou materiálovou úlohu (např. extrakce dat) MatBERT *drtivě překonává* obecné modely, protože již "rozumí" našemu specializovanému jazyku.

Kapitola 5: Závěr a výhled na příští týden

Shrnutí klíčových poznatků

Dnes jsme uzavřeli náš přehled základních architektur hlubokého učení tím, že jsme se vypořádali s *třetím pilířem dat*: sekvencemi.

1. Viděli jsme "klasický" přístup (RNN) a pochopili jeho fundamentální limity: neschopnost učit se dlouhodobé závislosti (mizející gradienty) a, což je ještě důležitější, *nemožnost paralelizace* (sekvenční bottleneck).

2. Představili jsme moderní řešení (Transformery), které nahrazuje rekurenci mechanismem **Self-Attention**. Tento přístup je *paralelizovatelný* (rychlý trénink) a má *přímý přístup* k celému kontextu (cesta signálu O(1)), čímž řeší oba problémy RNN.

3. Naučili jsme se klíčovou strategii, která pohání celou moderní AI: **Transfer Learning** (Předtrénování a Dolaďování).

4. Viděli jsme přímou aplikaci v naší doméně: využití ekosystému **Hugging Face** a doménově specifických modelů jako **MatBERT** k automatickému získávání znalostí z vědecké literatury.

Navázání na P12: Dvě nezodpovězené otázky

Náš arzenál modelů (CNN pro mřížky, GNN pro grafy, Transformery pro sekvence) je nyní téměř kompletní. Přesto všechny tyto modely sdílejí dvě zásadní slabiny, které jsou pro vědecké a inženýrské aplikace kritické.

Otázka 1: Problém "černé skříňky" a fyziky "Naše modely (CNN, GNN, Transformery) jsou černé skříňky řízené čistě daty. Trénujeme je, aby minimalizovaly chybu na datech. Co když ale *známe* fyzikální zákony, které v systému musí platit? (Např. zákon zachování hmoty, difuzní rovnice, Navier-Stokesovy rovnice).". Jak můžeme model *přinutit*, aby tyto zákony respektoval, místo abychom jen *doufali*, že se je naučí z dat? (Jak bylo naznačeno v přednášce 10 , GNN to dělají částečně přes invariantní architekturu, ale co když máme přímo rovnici?)

Otázka 2: Problém kvantifikace nejistoty "Naše modely dávají *bodové odhady* (point estimates). Predikují jedno číslo: 'pevnost je 1200 MPa'. Ale v inženýrství je predikce bez 'error baru' (chybové meze) téměř bezcenná." Jak moc si je model 'jistý' svou predikcí? Jak získáme zpět onu kvantifikaci nejistoty, kterou jsme měli například u Gaussovských procesů (v dřívější přednášce), ale ztratili ji u hlubokých neuronových sítí?.

Odpověď (Příští týden)

Příští týden se podíváme na dvě pokročilá témata, která se přesně těmito problémy zabývají a představují absolutní špičku výzkumu na pomezí AI a fyzikálních věd:

1. **Fyzikálně informované neuronové sítě (PINNs):** Naučíme se, jak integrovat parciální diferenciální rovnice (PDEs) přímo do *ztrátové funkce* neuronové sítě, a tím ji donutit, aby její řešení odpovídalo známé fyzice.

2. **Bayesovské hluboké učení (BNN):** Prozkoumáme modely, které se místo pevných hodnot vah (jak jsme viděli v přednášce 8) učí *pravděpodobnostní distribuce* pro každou váhu. To jim umožňuje kvantifikovat svou vlastní *nejistotu* v predikcích.

Works cited

1. Understanding Rnn | Illustrated Guide To Recurrent Neural ..., https://www.webhashtech.com/understanding-rnn-illustrated-guide-to-recurrent-neural-networks-understanding-the-intuition/ 2. Recurrent Neural Networks | CAIS++, https://caisplusplus.usc.edu/curriculum/neural-network-flavors/recurrent-neural-networks 3. Understanding Recurrent Neural Networks (RNNs) with an Intuitive Diagram - Medium, https://medium.com/@hamipirzada/understanding-recurrent-neural-networks-rnns-with-an-intuitive-diagram-3c556c7cfb80 4. What is a Recurrent Neural Network (RNN)? - IBM, https://www.ibm.com/think/topics/recurrent-neural-networks 5. Understanding the Vanishing and Exploding Gradient Problems in ..., https://medium.com/@helloitsdaksh007/understanding-the-vanishing-and-exploding-gradient-problems-in-rnns-7621efd97605 6. Vanishing and Exploding Gradients in Deep Neural Networks - Analytics Vidhya, https://www.analyticsvidhya.com/blog/2021/06/the-challenge-of-vanishing-exploding-gradients-in-deep-neural-networks/ 7. Vanishing and Exploding Gradients Problems in Deep Learning - GeeksforGeeks, https://www.geeksforgeeks.org/deep-learning/vanishing-and-exploding-gradients-problems-in-deep-learning/ 8. Co je problém mizejícího gradientu? - Akademie EITCA, https://cs.eitca.org/um%C4%9Bl%C3%A1-inteligence/eitc-ai-gcml-google-cloudov%C3%A9-strojov%C3%A9-u%C4%8Den%C3%AD/prvn%C3%AD-kroky-ve-strojov%C3%A9m-u%C4%8Den%C3%AD/hlubok%C3%A9-neuronov%C3%A9-s%C3%ADt%C4%9B-a-odhady/co-je-probl%C3%A9m-mizej%C3%ADc%C3%ADho-gradientu/ 9. What is RNN? - Recurrent Neural Networks Explained - AWS, https://aws.amazon.com/what-is/recurrent-neural-network/ 10. Why Recurrent Neural Networks (RNNs) Dominate Sequential Data Analysis - Shelf.io, https://shelf.io/blog/recurrent-neural-networks/ 11. 3 Pitfalls of RNNs and Why Transformers Rule the World | by Benjamin Bodner - Medium, https://medium.com/@benjybo7/3-pitfalls-of-rnns-and-why-transformers-rule-the-world-4661be4a1256 12. Gated Recurrent Unit Networks - Machine Learning - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/gated-recurrent-unit-networks/ 13. How LSTMs Solve the Vanishing Gradient Problem in Sequential ...,

https://ravjot03.medium.com/how-lstms-solve-the-vanishing-gradient-problem-in-sequential-data-b786eec3966f 14. Unlocking the Power of LSTM, GRU, and the Struggles of RNN in Managing Extended Sequences | by Murad Ahmad | Medium, https://medium.com/@muradatcorvit23/unlocking-the-power-of-lstm-gru-and-the-struggles-of-rnn-in-managing-extended-sequences-05879b6899d3 15. How does LSTM prevent the vanishing gradient problem? - Cross Validated, https://stats.stackexchange.com/questions/185639/how-does-lstm-prevent-the-vanishing-gradient-problem 16. RNN vs LSTM vs GRU vs Transformers - GeeksforGeeks, https://www.geeksforgeeks.org/deep-learning/rnn-vs-lstm-vs-gru-vs-transformers/ 17. RNN vs. LSTM vs. GRU: A Comprehensive Guide to Sequential Data Modeling - Medium, https://medium.com/@hassaanidrees7/rnn-vs-lstm-vs-gru-a-comprehensive-guide-to-sequential-data-modeling-03aab16647bb 18. An Intuitive Introduction to Transformers - Paperspace Blog, https://blog.paperspace.com/attention-is-all-you-need-the-components-of-the-transformer/ 19. Transformer (deep learning) - Wikipedia, https://en.wikipedia.org/wiki/Transformer_(deep_learning) 20. Self-Attention in Transformers: A Deep Dive | by Manish Negi | Medium, https://medium.com/@manishnegi101/self-attention-in-transformers-a-deep-dive-ec1d7eadc390 21. machine learning - Why does the transformer do better than RNN ..., https://ai.stackexchange.com/questions/20075/why-does-the-transformer-do-better-than-rnn-and-lstm-in-long-range-context-depen 22. Self-Attention Networks: Transformers, https://www1.se.cuhk.edu.hk/~seem5680/lecture/self-attention-transformer-2021spring.pdf 23. Why Transformers are better than LSTM and RNN | by Shridhar ..., https://medium.com/@shridharpawar77/why-transformers-are-better-than-lstm-and-rnn-6a9dbc51b178 24. Transformer vs RNN in NLP: A Comparative Analysis - Appinventiv, https://appinventiv.com/blog/transformer-vs-rnn/ 25. GPT-2 - Wikipedia, https://en.wikipedia.org/wiki/GPT-2 26. GPT-2 Detailed Model Architecture - Medium, https://medium.com/@hsinhungw/gpt-2-detailed-model-architecture-6b1aad33d16b 27. The Illustrated GPT-2 (Visualizing Transformer Language Models ..., https://jalammar.github.io/illustrated-gpt2/ 28. Transfer Learning with Fine-Tuning in NLP - GeeksforGeeks, https://www.geeksforgeeks.org/nlp/transfer-learning-and-fine-tuning-in-nlp/ 29. Transfer learning and fine-tuning | TensorFlow Core, https://www.tensorflow.org/tutorials/images/transfer_learning 30. Self-Supervised Learning - Part 1: Simple and intuitive introduction for beginners, https://insights.willogy.io/self-supervised-learning-part-1-simple-and-intuitive-introduction-for-beginners/ 31. NLP meets Materials Science: Quantifying the presentation of materials data in scientific literature - ChemRxiv, https://chemrxiv.org/engage/chemrxiv/article-details/658e4b29e9ebbb4db9f4dc8c 32. Revolutionizing Materials Science with NLP: Introducing MatSci-NLP and HoneyBee | Mila, https://mila.quebec/en/article/revolutionizing-materials-science-with-nlp-introducing-matsci-nlp-and-honeybee 33. Agent-based learning of materials datasets from the scientific literature - Digital Discovery (RSC Publishing) DOI:10.1039/D4DD00252K, https://pubs.rsc.org/en/content/articlehtml/2024/dd/d4dd00252k 34. Total noob's intro to Hugging Face Transformers, https://huggingface.co/blog/noob_intro_transformers 35. How to Use the Hugging Face Transformer Library - freeCodeCamp, https://www.freecodecamp.org/news/hugging-face-transformer-library-overview/ 36. Transformers - Hugging Face, https://huggingface.co/docs/transformers/index 37. Introduction to Hugging Face Transformers - GeeksforGeeks, https://www.geeksforgeeks.org/artificial-intelligence/Introduction-to-hugging-face-transformers/

38. MaterialBERT for Natural Language Processing of Materials ..., https://jxiv.jst.go.jp/index.php/jxiv/preprint/download/119/677/509 39. MaterialBERT for Natural Language Processing of Materials Science Texts - MDR, https://mdr.nims.go.jp/datasets/936f7bdd-e1eb-4100-929b-db71a00f8df9?locale=en 40. From Tokens to Materials: Leveraging Language Models for ..., https://arxiv.org/abs/2410.16165 41. MaterialBERT for natural language processing of materials science ..., https://www.tandfonline.com/doi/full/10.1080/27660400.2022.2124831 42. pranav-s/MaterialsBERT - Hugging Face, https://huggingface.co/pranav-s/MaterialsBERT 43. What Are Physics-Informed Neural Networks (PINNs)? - MATLAB ..., https://www.mathworks.com/discovery/physics-informed-neural-networks.html 44. Understanding Bayesian Neural Networks - Code B, https://code-b.dev/blog/bayesian-neural-networks 45. A first insight into Bayesian Neural Networks (BNNs) | by gabriel costa - Medium, https://medium.com/@costaleirbag/a-first-insight-into-bayesian-neural-networks-bnn-c767551e9526 46. Physics Informed Neural Networks (PINNs) in PhysicsNeMo Sym - NVIDIA Docs Hub, https://docs.nvidia.com/physicsnemo/latest/physicsnemo-sym/user_guide/theory/phys_informed.html 47. Introduction to Physics-informed Neural Networks | by Mario ..., https://medium.com/data-science/solving-differential-equations-with-neural-networks-afdcf7b8bcc4